

微處理器系統個人實驗報告 - Lab 09

實驗主題：多功能中央處理器擴充實作 (Multi-Function CPU Extension)

班級：資工二

學號：113590017 113590051

姓名：黃楷程 楊承諭

組別：第 19 組

組員：楊承諭、黃楷程

實驗日期：2026年6月17日

一、實驗內容與硬體規格

本次實驗 (Lab 09) 是在 Lab 08 簡易 CPU 的架構基礎上進行核心硬體的大幅擴充。我們維持了 4 個 8 位元通用暫存器 (R0~R3) 與米利型 (Mealy Machine) 有限狀態機控制單元，但引入了功能完整的**算術邏輯單元 (ALU)**，讓處理器從原本只能做資料搬移，升級為具備完整運算能力的 8 位元多功能中央處理器。

本系統採用 16 位元的指撥開關 SW[15:0] 作為硬體指令集 (ISA) 的輸入，具體支援的 8 條硬體指令與 4-bit 操作碼 (Opcode) 規格如下：

1. 資料搬移指令：

- 0000 : **LOAD Rs, Data** → 將 8 位元立即值 (Data) 鎖存至目的暫存器 Rs。
- 0001 : **MOVE Rs, Rt** → 將來源暫存器 Rt 的資料透過內部匯流排 (Bus) 複製到 Rs。

2. 算術邏輯運算指令 (ALU 擴充)：

- 0010 : **ADD Rs, Rt** → 計算兩暫存器相加 ($R_s + R_t$)，結果存回 Rs。

- 0011 : **AND Rs, Rt** → 對兩暫存器進行按位元 (Bitwise) 邏輯與運算，結果存回 Rs。
- 0100 : **SLT Rs, Rt** → 檢查是否「小於則置位」 (Set on Less Than) 。若 $R_s < R_t$ ，則結果為 1，否則為 0，並存回 Rs。
- 0101 : **SUB(A-B) Rs, Rt** → 執行減法運算 ($R_s - R_t$)，結果存回 Rs。
- 1000 : **DIV Rs, Rt** → 執行 8 位元整數除法運算 (R_s / R_t)，結果存回 Rs。
- 1001 : **SUB(B-A) Rs, Rt** → 執行反向減法運算 ($R_t - R_s$)，結果存回 Rs。

二、硬體週邊配置與腳位對照表

埠口名稱 (Port)	資料方向	位元寬度	對應板載硬體週邊 (Component)	FPGA 晶片腳位 (Pin)
KEY[0]	Input	1	觸發按鍵 KEY0 (手動觸發中央處理器系統時脈)	PIN_M23
SW[7..0]	Input	8	指撥開關 SW7 ~ SW0 (8位元平行立即值資料輸入)	PIN_AB26 至 PIN_AB28
SW[11..8]	Input	4	指撥開關 SW11 ~ SW8 (4位元微指令操作碼輸入)	PIN_AC25 等週邊管腳
SW[13..12]	Input	2	指撥開關 SW13 ~ SW12 (目的暫存器 Rs 選擇)	依 CPU.qsf 物理配置
SW[15..14]	Input	2	指撥開關 SW15 ~ SW14 (來源暫存器 Rt 選擇)	依 CPU.qsf 物理配置
HEX1, HEX0	Output	14	七段顯示器 HEX1, HEX0 (即時顯示內部匯流排 Bus / ALU 輸出)	依 .qsf 綁定配置
HEX3, HEX2	Output	14	七段顯示器 HEX3, HEX2 (即時顯示目的暫存器 Rs 當前數值)	依 .qsf 綁定配置
HEX5, HEX4	Output	14	七段顯示器 HEX5, HEX4 (即時顯示來源暫存器 Rt 當前數值)	依 .qsf 綁定配置

三、核心 VHDL 原始碼分析

本實驗的多功能 CPU 核心架構採用了高效的控制與資料路徑分離設計，底層透過組合邏輯即時計算 ALU 運算與內部匯流排值，並於時脈正緣同步更新暫存器陣列。其核心 VHDL 程式碼結構如下：

VHDL

```
--
=====

-- Lab 09: Multi-Function CPU Implementation (Core Architecture)

--
=====

library ieee;

use ieee.std_logic_1164.all;

use ieee.std_logic_unsigned.all;

entity CPU is

    port (

        SW : in  std_logic_vector(15 downto 0); -- 指撥開關指令輸入

        KEY : in  std_logic_vector(0 downto 0); -- 時脈輸入 KEY[0]

        HEX0, HEX1, HEX2, HEX3, HEX4, HEX5 : out std_logic_vector(6 downto
0) -- 七段顯示器

    );

end entity CPU;
```

architecture Behavioral of CPU is

```
-- 暫存器宣告

signal R0, R1, R2, R3 : std_logic_vector(7 downto 0) := (others => '0');

signal Bus_val, Rs_val, Rt_val : std_logic_vector(7 downto 0);


-- 指令別名映射 (SW[15:0])

alias Data    : std_logic_vector(7 downto 0) is SW[7:0];
alias Opcode  : std_logic_vector(3 downto 0) is SW[11:8];
alias Rs_sel  : std_logic_vector(1 downto 0) is SW[13:12];
alias Rt_sel  : std_logic_vector(1 downto 0) is SW[15:14];

begin

    -- 暫存器多工器解碼 (Mux)

    Rs_val <= R0 when Rs_sel = "00" else R1 when Rs_sel = "01" else R2 when
Rs_sel = "10" else R3;

    Rt_val <= R0 when Rt_sel = "00" else R1 when Rt_sel = "01" else R2 when
Rt_sel = "10" else R3;


    --
=====
=====

    -- Combinational Logic: ALU / Bus Value Selector (即時運算)

    --
=====
=====
```

```

process(Opcode, Data, Rs_val, Rt_val)

begin

  case Opcode is

    when "0000" => Bus_val <= Data;          -- LOAD

    when "0001" => Bus_val <= Rt_val;         -- MOVE

    when "0010" => Bus_val <= Rs_val + Rt_val; -- ADD

    when "0011" => Bus_val <= Rs_val and Rt_val; -- AND

    when "0100" =>                               -- SLT

      if (Rs_val < Rt_val) then Bus_val <= "00000001";

      else Bus_val <= "00000000"; end if;

    when "0101" => Bus_val <= Rs_val - Rt_val; -- SUB (A-B)

    when "1000" =>                               -- DIV (Integer Division)

      if (Rt_val /= "00000000") then Bus_val <= Rs_val / Rt_val;

      else Bus_val <= (others => '1'); end if; -- 避免除以 0 錯誤

    when "1001" => Bus_val <= Rt_val - Rs_val; -- SUB (B-A)

    when others => Bus_val <= (others => '0');

  end case;

end process;


--
=====
=====

-- Sequential Logic: Synchronous Register Write Back (時脈正緣寫回)

```

```

--
=====

process(clk)

begin

    if rising_edge(clk) then

        if (Opcode = "0000" or Opcode = "0001" or Opcode = "0010" or
Opcode = "0011" or

            Opcode = "0100" or Opcode = "0101" or Opcode = "1000" or
Opcode = "1001") then

            case Rs_sel is

                when "00"  => R0 <= Bus_val;

                when "01"  => R1 <= Bus_val;

                when "10"  => R2 <= Bus_val;

                when others => R3 <= Bus_val;

            end case;

        end if;

    end if;

end process;

-- 省略 hex_to_7seg 的解碼電路指定 (HEX0~HEX5 驅動)

end architecture Behavioral;

```

三、 實驗心得與深度思考

理解算術邏輯單元 (ALU) 與內部匯流排的整合： 在這次 Lab 09 裡面，CPU 的架構變得更像一個真正的處理器。我們把前幾次實驗學到的算術運算（如加、減、除法、布林邏輯）做成一個龐大的組合邏輯 ALU 區塊，並透過匯流排 (Bus) 與暫存器陣列串接。控制單元透過讀取 `SW[11:8]` 的 4-bit 操作碼，即時決定 Bus 究竟要抓取哪一種運算結果。這種資料路徑 (Data Path) 的動態切換，讓我真正體會到計算機組織結構的設計精髓。

同步鎖存與組合邏輯 Live 輸出的工程考量： 在分析 VHDL 程式碼時，我注意到一組很有意思的設計：七段顯示器 `HEX1/HEX0` 顯示的 Bus 數值是即時 (live / combinational) 呈現的，它會隨著指撥開關切換 Opcode 立即產生變化；但真正要將這個運算結果寫入目的暫存器 `Rs` 時，必須等到手動時脈 `KEY[0]` 的正緣觸發。這種「組合邏輯即時運算、時序邏輯同步鎖存」的設計，大幅提升了資料傳輸的穩定度，完美過濾了開關切換時的雜訊毛邊。

指令集架構 (ISA) 微架構設計的收穫： 從 Lab 08 的 2 條指令擴充到這次的 8 條指令，雖然功能變複雜了，但因為底層暫存器陣列和控制骨架設計得很模組化，我們只需要在 `process` 的 `case Opcode` 裡面增加對應的硬體描述語句（像是加上加法 +、減法 - 或除法 /）就能完成擴充。這讓我深刻體會到，現代處理器（如 RISC-V 或 x86）不論功能再怎麼強大，其底層核心的 ASM（算法狀態機）與模組化擴充觀念，跟我們在實驗室裡實作的 Simple CPU 完全是一脈相承的。

四、 工作分配表

學號	姓名	比重	工作項目
113590051	楊承諭	65%	小組專案製作與測試
113590017	黃楷程	35%	小組報告製作